

ALL from DS

Ответы на вопросы экзамена по машинному обучению

Основная программа

1. Machine Learning problem statement. Regression, Classification, examples.

Машинное обучение (МО) — это область искусственного интеллекта, изучающая методы построения моделей, которые способны обучаться на данных и выполнять задачи без явного программирования. Основная цель — извлечь знания (закономерности) из данных.

Постановка задачи supervised learning (обучение с учителем):

Имеется выборка $L = \{(x_i, y_i)\}_{i=1}^n$, где $x_i \in \mathbb{R}^p$ — вектор признаков объекта, y_i — целевая переменная (ответ). Требуется найти функцию $f: X \rightarrow Y$, которая наилучшим образом предсказывает y по x .

Основные типы задач:

- **Регрессия (Regression):** $y \in \mathbb{R}$ (непрерывная величина). Пример: предсказание цены дома по его площади, количеству комнат, району.
- **Классификация (Classification):** $y \in \{C_1, \dots, C_k\}$ (дискретная величина, метка класса). Пример: определение типа ириса (setosa, versicolor, virginica) по размерам чашелистика и лепестка.

Unsupervised learning (обучение без учителя): Имеются только данные $\{x_i\}$, целевые переменные отсутствуют. Задачи: кластеризация, снижение размерности, поиск аномалий. Пример: группировка клиентов по истории покупок.

2. How to measure quality in classification: accuracy, balanced accuracy, precision, recall, f1-score, ROC-AUC, multiclass extensions.

Accuracy (точность): доля правильно классифицированных объектов.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Не подходит для несбалансированных классов.

Balanced accuracy: среднее арифметическое accuracy для каждого класса. Учитывает дисбаланс.

Precision (точность): доля правильно предсказанных положительных объектов среди всех объектов, предсказанных как положительные.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (полнота): доля правильно предсказанных положительных объектов среди всех действительно положительных объектов.

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-score (F-мера): гармоническое среднее precision и recall.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Для регулировки важности precision и recall используется F_β -мера.

ROC-кривая (Receiver Operating Characteristic): график зависимости TPR (True Positive Rate, то же что recall) от FPR (False Positive Rate) при варьировании порога бинаризации вероятностей.

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}$$

ROC-AUC (Area Under Curve): площадь под ROC-кривой. Характеризует способность модели разделять классы. AUC = 0.5 — случайное угадывание, AUC = 1 — идеальное разделение.

Мультиклассовые расширения: Для многоклассовой классификации метрики вычисляются для каждого класса отдельно, затем агрегируются:

- **Micro-averaging:** вычисление метрик с учётом глобальных TP, FP, FN.
- **Macro-averaging:** усреднение метрик по классам.
- **Weighted averaging:** усреднение с весами, пропорциональными размеру классов.

3. How to measure quality in regression: MSE, MAE, R2.

MSE (Mean Squared Error, средняя квадратичная ошибка):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Сильно штрафует большие ошибки, дифференцируема.

MAE (Mean Absolute Error, средняя абсолютная ошибка):

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Менее чувствительна к выбросам, чем MSE.

R² (коэффициент детерминации): показывает, насколько модель лучше предсказывает данные по сравнению с константным предсказанием (средним значением).

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Может быть отрицательным, если модель предсказывает хуже среднего.

Другие метрики: RMSE (корень из MSE), MAPE (средняя абсолютная процентная ошибка), SMAPE (симметричная MAPE).

4. Maximum likelihood estimation, how is it related to regression and classification

Метод максимального правдоподобия (Maximum Likelihood Estimation, MLE): метод оценки параметров модели, который максимизирует функцию правдоподобия. Функция правдоподобия $L(\theta|X, Y) = P(X, Y|\theta)$ — это вероятность наблюдать данные при заданных параметрах θ . В предположении i.i.d.:

$$L(\theta|X, Y) = \prod_{i=1}^n P(x_i, y_i|\theta)$$

На практике максимизируют логарифмическое правдоподобие:

$$\log L(\theta|X, Y) = \sum_{i=1}^n \log P(x_i, y_i|\theta) \rightarrow \max_{\theta}$$

Связь с регрессией: В линейной регрессии минимизация MSE эквивалентна MLE, если ошибки ϵ_i независимы и нормально распределены с нулевым средним и постоянной дисперсией.

Связь с классификацией: В логистической регрессии минимизация логистического лосса эквивалентна MLE для биномиального распределения. Для наивного байесовского классификатора параметры (априорные вероятности классов и распределения признаков) также оцениваются через MLE (частоты).

5. Naive bayesian classifier, how does it work

Наивный байесовский классификатор — алгоритм классификации, основанный на теореме Байеса и предположении о **наивной** (условной) независимости признаков при данном классе.

Алгоритм:

1. Дано: объект $x = (x^1, \dots, x^p)$, классы $C_1, \dots, C_k$.
2. Для каждого класса C_k вычисляется апостериорная вероятность:

$$P(y = C_k | x) = \frac{P(x | y = C_k) P(y = C_k)}{P(x)}$$

3. В силу **наивного** предположения:

$$P(x | y = C_k) = \prod_{j=1}^p P(x^j | y = C_k)$$

4. Класс объекта предсказывается как:

$$\hat{y} = \arg \max_{C_k} P(y = C_k | x) = \arg \max_{C_k} P(y = C_k) \prod_{j=1}^p P(x^j | y = C_k)$$

(знаменатель $P(x)$ не зависит от класса и не влияет на $\arg \max$).

Оценка вероятностей:

- $P(y = C_k)$ — частота класса C_k в обучающей выборке.
- $P(x^j | y = C_k)$ — зависит от типа признака:
- Категориальный: частота значения x^j в классе C_k .
- Непрерывный: предполагается параметрическое распределение (например, нормальное), параметры которого оцениваются по выборке.

Почему "наивный"? Предположение о независимости признаков редко выполняется на практике, но модель часто работает хорошо благодаря своей простоте и низкой вычислительной сложности.

6. K-nearest neighbours classifier, how does it work

k ближайших соседей (k-Nearest Neighbours, kNN) — непараметрический алгоритм классификации (и регрессии), основанный на гипотезе компактности: похожие объекты лежат в одном классе.

Алгоритм классификации:

1. Запоминается вся обучающая выборка $\{(x_i, y_i)\}$.
2. Для нового объекта x вычисляются расстояния до всех объектов обучающей выборки в пространстве признаков.
3. Выбираются k объектов с наименьшими расстояниями.
4. Класс x определяется как наиболее частый класс среди этих k соседей (голосование).

Алгоритм регрессии: Вместо голосования вычисляется среднее (или взвешенное среднее) целевых значений соседей.

Важные аспекты:

- **Выбор метрики расстояния:** Евклидово, Манхэттенское, косинусное и др.
- **Выбор k (гиперпараметр):**
- Малое k : модель чувствительна к шуму (переобучение).
- Большое k : сглаживание, но возможна недооценка локальных закономерностей.
- **Масштабирование признаков** обязательно, т.к. алгоритм чувствителен к масштабу.
- **Взвешенный kNN:** вес соседа обратно пропорционален расстоянию до x .

Недостатки: Высокие вычислительные затраты на предсказание (нужно вычислять расстояния до всех обучающих объектов), чувствительность к размерности пространства (проклятие размерности).

7. Linear regression. Problem statement for the MSE loss function case. Analytical solution. Gauss-Markov theorem. Gradient approach in linear regression.

Постановка задачи линейной регрессии:

Дана выборка $\{(x_i, y_i)\}_{i=1}^n$, где $x_i \in \mathbb{R}^p$, $y_i \in \mathbb{R}$. Модель: $f_w(x) = w_0 + w_1x_1 + \dots + w_px_p = x^T w$, где $w = (w_0, w_1, \dots, w_p)^T$, а в вектор признаков добавлена единица: $x = (1, x_1, \dots, x_p)^T$.

Минимизация MSE:

$$Q(w) = \frac{1}{n} \sum_{i=1}^n (y_i - x_i^T w)^2 = \frac{1}{n} \|Y - Xw\|^2 \rightarrow \min_w$$

где $X \in \mathbb{R}^{n \times (p+1)}$ — матрица объекты-признаки, $Y \in \mathbb{R}^n$ — вектор ответов.

Аналитическое решение (нормальное уравнение):

Приравнивая градиент к нулю, получаем:

$$w^* = (X^T X)^{-1} X^T Y$$

Условие существования: матрица $X^T X$ должна быть обратимой (признаки линейно независимы).

Теорема Гаусса-Маркова: Если ошибки $\epsilon_i = y_i - x_i^T w$ удовлетворяют условиям:

1. $\mathbb{E}[\epsilon_i] = 0$ (несмещённость),
 2. $\text{Var}(\epsilon_i) = \sigma^2 < \infty$ (гомоскедастичность),
 3. $\text{Cov}(\epsilon_i, \epsilon_j) = 0$ при $i \neq j$ (некоррелированность),
- то оценка w^* , полученная методом наименьших квадратов (минимизацией MSE), является **наилучшей линейной несмещённой оценкой (BLUE)**: она имеет минимальную дисперсию среди всех линейных несмещённых оценок.

Градиентный подход: При больших n или p аналитическое решение может быть вычислительно сложным. Альтернатива — итеративные методы:

$$w^{(t+1)} = w^{(t)} - \eta \cdot \nabla Q(w^{(t)}), \quad \nabla Q(w) = -\frac{2}{n} X^T (Y - Xw)$$

где η — скорость обучения.

8. Regularization in linear models: L1 and L2, their properties. Probabilistic interpretation.

Регуляризация — метод борьбы с переобучением путём добавления штрафа за сложность модели (большие значения весов) к функции потерь.

L2-регуляризация (Ridge Regression):

$$Q_{L2}(w) = \|Y - Xw\|^2 + \lambda \|w\|_2^2 \rightarrow \min_w$$

где $\|w\|_2^2 = \sum_{j=0}^p w_j^2$, $\lambda > 0$ — гиперпараметр регуляризации.

- **Решение:** $w^{ridge} = (X^T X + \lambda I)^{-1} X^T Y$. Добавление λI делает задачу устойчивой даже при вырожденной $X^T X$.
- **Свойства:** Веса равномерно уменьшаются (сжимаются к нулю), но не обнуляются. Помогает бороться с мультиколлинеарностью.

L1-регуляризация (LASSO — Least Absolute Shrinkage and Selection Operator):

$$Q_{L1}(w) = \|Y - Xw\|^2 + \lambda \|w\|_1 \rightarrow \min_w$$

где $\|w\|_1 = \sum_{j=0}^p |w_j|$.

- **Свойства:** Приводит к обнулению части весов (разреженное решение), выполняя таким образом **отбор признаков**. Не имеет аналитического решения в общем случае, решается итеративными методами.

ElasticNet: Комбинация L1 и L2: $\|Y - Xw\|^2 + \lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2$.

Вероятностная интерпретация:

- **L2-регуляризация** соответствует априорному распределению весов $w_j \sim \mathcal{N}(0, \tau^2)$ (нормальное распределение).
- **L1-регуляризация** соответствует априорному распределению Лапласа: $w_j \sim \text{Laplace}(0, b)$. Максимизация апостериорной вероятности (MAP) при таких априорных распределениях приводит к соответствующим регуляризованным задачам.

9. Logistic regression. Equivalence of MLE approach and logistic loss minimization.

Логистическая регрессия — линейная модель для **бинарной** классификации, предсказывающая вероятность принадлежности к классу 1.

Модель: Вероятность того, что объект x принадлежит классу 1, моделируется с помощью **сигмоидной (логистической) функции**:

$$P(y = 1|x, w) = \sigma(x^T w) = \frac{1}{1 + e^{-x^T w}}$$

где $x^T w = w_0 + w_1 x_1 + \dots + w_p x_p$.

Логистический лосс (log loss, cross-entropy loss):

$$L(w) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\sigma(x_i^T w)) + (1 - y_i) \log(1 - \sigma(x_i^T w))]$$

где $y_i \in \{0, 1\}$.

Эквивалентность MLE и минимизации логистического лосса:

Рассмотрим задачу MLE. Правдоподобие выборки:

$$L(w|X, Y) = \prod_{i=1}^n P(y_i|x_i, w) = \prod_{i=1}^n \sigma(x_i^T w)^{y_i} (1 - \sigma(x_i^T w))^{1-y_i}$$

Логарифмическое правдоподобие:

$$\log L(w) = \sum_{i=1}^n [y_i \log \sigma(x_i^T w) + (1 - y_i) \log(1 - \sigma(x_i^T w))]$$

Максимизация логарифмического правдоподобия **эквивалентна** минимизации логистического лосса (с точностью до знака и нормировки). Таким образом, обучение логистической регрессии методом MLE сводится к минимизации логистического лосса.

10. Multiclass classification. One-vs-one, one-vs-all, their properties.

Мультиклассовая классификация — задача классификации с числом классов $K > 2$.

Линейные модели изначально бинарны, поэтому требуются стратегии расширения.

One-vs-All (OvA, One-vs-Rest):

- Строится K бинарных классификаторов.
- k -й классификатор обучается отличать класс C_k от **всех остальных** классов, объединённых в один "negative" класс.
- Для нового объекта x запускаются все K классификаторов, выбирается класс с наибольшим значением **решающей функции** (например, вероятности или отступа).
- **Плюсы:** Требуется построить только K моделей.
- **Минусы:** Дисбаланс классов в задачах (один класс против многих), возможны "серые зоны", где несколько классификаторов дают положительный ответ.

One-vs-One (OvO):

- Строится $\frac{K(K-1)}{2}$ бинарных классификаторов — по одному для каждой пары классов (C_i, C_j) .
- Каждый классификатор обучается только на объектах из двух классов C_i и C_j .
- Для нового объекта x каждый классификатор "голосует" за один из двух своих классов. Итоговый класс — тот, который набрал наибольшее число голосов.
- **Плюсы:** Каждый классификатор обучается на более сбалансированной и простой задаче.
- **Минусы:** Требуется построить $O(K^2)$ моделей, что может быть затратно при больших K .

Выбор стратегии: Зависит от задачи, числа классов, размера выборки. OvA более популярна из-за меньшего числа моделей. Некоторые алгоритмы (например, SVM) часто используют OvO.

11. Support vector machine. Optimization problem for SVM. Kernel trick. Kernel properties.

Метод опорных векторов (SVM) — алгоритм для классификации (и регрессии), целью которого является нахождение гиперплоскости, максимально удалённой от объектов обоих классов (**максимизация зазора, margin**).

Оптимизационная задача для линейно разделимого случая:

Дана выборка $\{(x_i, y_i)\}$, $y_i \in \{-1, +1\}$. Гиперплоскость: $w^T x + b = 0$.

Желательно, чтобы для всех объектов выполнялось: $y_i(w^T x_i + b) \geq 1$.

Ширина зазора пропорциональна $\frac{2}{\|w\|}$. Максимизация зазора эквивалентна минимизации нормы вектора весов:

$$\begin{cases} \frac{1}{2} \|w\|^2 \rightarrow \min_{w,b} \\ y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, n \end{cases}$$

Это задача квадратичного программирования.

Оптимизационная задача для линейно неразделимого случая (SVM с мягким зазором, soft-margin SVM):

Вводятся переменные ослабления (slack variables) $\xi_i \geq 0$, допускающие ошибки:

$$\begin{cases} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \rightarrow \min_{w,b,\xi} \\ y_i(w^T x_i + b) \geq 1 - \xi_i, \quad i = 1, \dots, n \\ \xi_i \geq 0 \end{cases}$$

где $C > 0$ — гиперпараметр, регулирующий компромисс между шириной зазора и числом ошибок.

Двойственная задача (Dual Problem): Через метод множителей Лагранжа задача сводится к двойственной, которая зависит только от скалярных произведений объектов:

$$\begin{cases} \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j (x_i^T x_j) \rightarrow \max_{\lambda} \\ 0 \leq \lambda_i \leq C, \quad \sum_{i=1}^n \lambda_i y_i = 0 \end{cases}$$

Решение: $w = \sum_{i=1}^n \lambda_i y_i x_i$. Опорные векторы — объекты с $\lambda_i > 0$.

Ядерный трюк (Kernel Trick): Если данные нелинейно разделимы, можно отобразить их в пространство более высокой размерности (признаковое пространство) $\phi : \mathbb{R}^p \rightarrow \mathcal{H}$, где они становятся линейно разделимыми. В двойственной задаче появляются скалярные произведения $\phi(x_i)^T \phi(x_j)$. **Ядро (Kernel)** — функция $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$, которая вычисляет скалярное произведение в пространстве $\mathcal{H}$, не вычисляя явно отображение ϕ .

Свойства ядер: Должны быть симметричными и положительно определёнными (удовлетворять условию Мерсера).

- **Линейное:** $K(x, x') = x^T x'$.
- **Полиномиальное:** $K(x, x') = (x^T x' + c)^d$.
- **RBF (Radial Basis Function, гауссово):** $K(x, x') = \exp(-\gamma \|x - x'\|^2)$.

Нелинейное SVM: Решается двойственная задача с заменой скалярного произведения на ядро $K(x_i, x_j)$. Решающая функция: $f(x) = \sum_{i=1}^n \lambda_i y_i K(x_i, x) + b$.

12. Principal component analysis. Relations to SVD. Eckart-Young theorem. How to apply PCA in practice.

Метод главных компонент (PCA) — линейный метод снижения размерности, который находит ортогональные направления максимальной дисперсии данных и проецирует данные на эти направления.

Постановка задачи: Дана матрица данных $X \in \mathbb{R}^{n \times p}$ (признаки центрированы). Требуется найти ортонормированные векторы $u_1, \dots, u_k$ (главные компоненты), такие что проекции данных на них имеют максимальную выборочную дисперсию.

Решение через сингулярное разложение (SVD):

Для центрированной матрицы X выполняется SVD: $X = U \Sigma V^T$, где $U \in \mathbb{R}^{n \times n}$, $V \in \mathbb{R}^{p \times p}$ — ортогональные матрицы, Σ — диагональная матрица сингулярных чисел $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0$.

- Столбцы V — это главные компоненты (собственные векторы ковариационной матрицы $X^T X$).
- Сингулярные числа связаны с дисперсией: $\text{Var}(X v_j) = \frac{\sigma_j^2}{n-1}$.
- Приведение к k размерностям: $X_k = U_k \Sigma_k V_k^T$, где U_k, V_k содержат первые k столбцов, Σ_k — первые k сингулярных чисел. Это наилучшая (в смысле нормы Фробениуса) аппроксимация ранга k исходной матрицы X .

Теорема Эккарта-Янга: Для любой матрицы X и любого целого $k < \text{rank}(X)$, матрица X_k , полученная усечением SVD до k наибольших сингулярных чисел, является решением задачи:

$$\min_{B: \text{rank}(B)=k} \|X - B\|_F$$

Практическое применение PCA:

1. **Предобработка данных:** Центрирование (вычитание среднего) и часто масштабирование (нормировка на стандартное отклонение), чтобы признаки с большей дисперсией не доминировали.
2. **Вычисление главных компонент:** Через SVD или собственные векторы ковариационной матрицы.
3. **Выбор числа компонент k :**
 - По доле объяснённой дисперсии: выбирают минимальное k , при котором сумма дисперсий первых k компонент превышает заданный порог (например, 95%).
 - По "локтя" на графике объяснённой дисперсии.
4. **Преобразование данных:** Новые признаки — проекции на главные компоненты: $T = XV_k$.
5. **Реконструкция:** $\tilde{X} = TV_k^T$. Ошибка восстановления $\|X - \tilde{X}\|_F$.

Применение: Сжатие данных, визуализация (проекция на 2-3 компоненты), удаление шума, ускорение обучения моделей.

13. Train, validation and test stages of model development. Overfitting problem, ways to detect it.

Этапы разработки модели:

1. **Обучение (Train):** На тренировочной выборке подбираются **параметры модели** (например, веса линейной регрессии) путём минимизации функции потерь.
2. **Валидация (Validation):** На валидационной выборке (отложенной от тренировочной) подбираются **гиперпараметры модели** (например, степень регуляризации λ , глубина дерева). Модель с разными гиперпараметрами обучается на train, оценивается на validation, выбираются лучшие гиперпараметры.
3. **Тестирование (Test):** На тестовой выборке (не использовавшейся ни на train, ни на validation) один раз оценивается итоговое качество модели с фиксированными параметрами и гиперпараметрами. Это оценка способности модели обобщаться на новые данные.

Переобучение (Overfitting): Модель слишком хорошо подстраивается под тренировочные данные (включая шум) и теряет способность обобщаться. Проявляется в большой разнице между качеством на train и validation/test.

Способы обнаружения переобучения:

- **Мониторинг кривых обучения:** График зависимости функции потерь (или метрики) на train и validation от числа итераций обучения (эпох) или сложности модели.
- **Признак переобучения:** Ошибка на train продолжает уменьшаться, а ошибка на validation начинает расти.
- **Признак недообучения (Underfitting):** Ошибки на train и validation высоки и близки друг к другу.
- **Использование валидационной выборки** для ранней остановки (early stopping) при росте ошибки на validation.
- **Сравнение производительности** на train и test: если accuracy на train близка к 100%, а на test значительно ниже — явный признак переобучения.

14. Validation strategies. Cross validation. Data leaks.

Стратегии валидации:

- **Hold-out validation:** Исходная выборка случайно разбивается на три части: train, validation, test. Недостаток: оценка может сильно зависеть от разбиения.
- **Кросс-валидация (Cross-Validation, CV):** Более стабильный метод.
- **K-fold CV:** Выборка разбивается на K непересекающихся частей (фолдов). $K - 1$ фолдов используются для обучения, 1 — для валидации. Процедура повторяется K раз, каждый раз в качестве валидационного используется новый фолд. Итоговая оценка — среднее по всем фолдам.
- **Стратифицированная K-fold CV:** В каждом фолде сохраняется распределение классов (важно для несбалансированных данных).
- **Leave-One-Out CV (LOOCV):** Частный случай, когда $K = n$ (каждый объект по очереди становится валидационным). Точная, но вычислительно дорогая.
- **Nested Cross-Validation:** Используется для одновременного подбора гиперпараметров и оценки качества. Внешний цикл для оценки, внутренний — для подбора гиперпараметров.

Утечка данных (Data Leak): Ситуация, когда информация из тестовой (или валидационной) выборки неявно используется при обучении модели, что приводит к завышенным и неадекватным оценкам качества.

Типичные причины:

- Предобработка данных (масштабирование, заполнение пропусков) проводится по всей выборке **до** её разделения. Правильно: вычислять параметры преобразования (среднее, дисперсию) только на тренировочной части, затем применять с этими параметрами к validation и test.
- Использование тестовой выборки для выбора модели или гиперпараметров.
- Временные ряды: использование будущих данных для предсказания прошлого.

Как избежать: Чёткое разделение данных на train/validation/test на самом раннем этапе, пайплайны с изоляцией шагов предобработки.

15. Bias-variance tradeoff.

Разложение ошибки: Ожидаемая ошибка модели на новом объекте x можно разложить на три составляющие:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{Смещение (Bias)}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{Разброс (Variance)}} + \underbrace{\sigma^2}_{\text{Неустраняемая ошибка}}$$

где $f(x)$ — истинная функция, $\hat{f}(x)$ — предсказание модели, σ^2 — дисперсия шума.

- **Смещение (Bias):** Ошибка, вызванная упрощением модели (например, попытка аппроксимировать нелинейную зависимость линейной моделью). Высокое смещение ведёт к **недообучению**.
- **Разброс (Variance):** Чувствительность модели к конкретной обучающей выборке (к шуму в данных). Высокий разброс ведёт к **переобучению**.

Компромисс (tradeoff): При увеличении сложности модели (например, степени полинома, глубины дерева) смещение обычно уменьшается, а разброс увеличивается. И наоборот, при упрощении модели смещение растёт, а разброс падает. Задача — найти оптимальную сложность, минимизирующую общую ошибку.

Связь с ансамблями:

- **Бэггинг (Bagging):** Усреднение предсказаний нескольких моделей, обученных на разных подвыборках, уменьшает **разброс**, не меняя смещение.
- **Бустинг (Boosting):** Последовательное обучение моделей, корректирующих ошибки предыдущих, уменьшает **смещение**, но может увеличивать разброс (требует осторожной регуляризации).

16. Decision tree construction procedure.

Дерево решений — модель, предсказывающая целевую переменную путём рекурсивного разбиения пространства признаков на области и присвоения каждой области константного значения (в листе).

Жадный алгоритм построения (рекурсивное деление):

1. **Начало:** Всё множество объектов R — корневой узел.
2. **Выбор наилучшего разбиения:** Для каждого признака j и каждого возможного порога t (или для категорий) вычисляется критерий качества разбиения $G(j, t)$. Выбирается пара (j^*, t^*) , дающая наилучшее значение критерия.
3. **Разбиение:** Текущий узел разбивается на два дочерних узла: $R_{\text{left}} = \{x \in R | x^{(j^*)} < t^*\}$ и $R_{\text{right}} = \{x \in R | x^{(j^*)} \geq t^*\}$.

4. **Рекурсия:** Шаги 2-3 повторяются для каждого дочернего узла, пока не выполнится **критерий остановки**.

Критерий качества разбиения: Оценивает, насколько разбиение увеличивает "однородность" дочерних узлов. Для узла R вводят меру неоднородности (impurity) $H(R)$. Качество разбиения:

$$G(j, t) = \frac{|R_{\text{left}}|}{|R|} H(R_{\text{left}}) + \frac{|R_{\text{right}}|}{|R|} H(R_{\text{right}})$$

Чем меньше G (чем однороднее дочерние узлы), тем лучше.

Критерии остановки:

- Достигнута максимальная глубина.
- Число объектов в узле меньше минимального.
- Улучшение критерия G меньше порога.
- Все объекты в узле принадлежат одному классу (для классификации) или имеют одинаковое значение (для регрессии).

Присвоение значения в листе:

- **Классификация:** Класс, наиболее часто встречающийся в листе (мода).
- **Регрессия:** Среднее значение целевой переменной объектов в листе.

17. Information criteria. Entropy criteria, Gini impurity.

Критерии информативности (меры неоднородности) для классификации:

- **Энтропия (Entropy):** $H(R) = - \sum_{k=1}^K p_k \log_2 p_k$, где p_k — доля объектов класса k в узле R .
- Максимальна при равномерном распределении классов ($p_k = 1/K$), равна 0, если все объекты одного класса.
- Основана на понятии информации Шеннона.
- **Критерий Джини (Gini impurity):** $H(R) = 1 - \sum_{k=1}^K p_k^2$.
- Также максимальна при равномерном распределении, равна 0 в чистом узле.
- Имеет вероятностную интерпретацию: вероятность ошибки, если случайно выбрать два объекта из узла и они окажутся разных классов.

Сравнение: Оба критерия похожи по поведению. Gini немного быстрее вычисляется, так как не требует вычисления логарифма. На практике разница в качестве построенных деревьев незначительна.

Для регрессии используются критерии, основанные на дисперсии целевой переменности в узле:

- **MSE:** $H(R) = \frac{1}{|R|} \sum_{i \in R} (y_i - \bar{y}_R)^2$, где $\bar{y}_R$ — среднее в узле.

- **MAE:** $H(R) = \frac{1}{|R|} \sum_{i \in R} |y_i - \text{median}(y_R)|$.

18. Ensembling methods. Bootstrap. Bagging.

Ансамблирование (Ensembling) — комбинирование прогнозов нескольких базовых моделей для получения более точного и устойчивого итогового прогноза.

Бутстрэп (Bootstrap): Статистический метод генерации новых выборок (бутстрэп-выборок) из исходной выборки путём случайного выбора объектов **с возвращением** (sampling with replacement). Каждая бутстрэп-выборка имеет тот же размер n , что и исходная. Некоторые объекты попадают в выборку несколько раз, некоторые — ни разу. Объекты, не попавшие в выборку, называются **out-of-bag (OOB)**.

Бэггинг (Bootstrap Aggregating):

1. Генерируется B бутстрэп-выборок из исходной обучающей выборки.
2. На каждой выборке независимо обучается своя базовая модель (например, дерево решений).
3. Итоговый прогноз — усреднение (для регрессии) или голосование (для классификации) прогнозов всех B моделей.

Эффект: Бэггинг уменьшает **разброс (variance)** ошибки модели, не влияя на **смещение (bias)**. Особенно эффективен для моделей с высокой дисперсией (неустойчивых), таких как глубокие деревья решений.

Обоснование: Если базовые модели независимы и несмещённые, то дисперсия среднего в B раз меньше дисперсии отдельной модели. На практике модели коррелированы, поэтому выигрыш меньше, но всё равно значителен.

19. Random Forest, Random subspace method.

Случайный лес (Random Forest) — ансамбль решающих деревьев, построенный с использованием бэггинга и дополнительного **внесения случайности** для уменьшения корреляции между деревьями.

Random Subspace Method (метод случайных подпространств): При построении каждого дерева на каждом шаге выбора наилучшего разбиения рассматривается не все множество признаков (p), а случайное подмножество размера m (часто $m = \sqrt{p}$ или $\log_2 p$). Это заставляет деревья быть более разнообразными.

Алгоритм Random Forest:

1. Для $b = 1$ до B :
 - Сгенерировать бутстрэп-выборку.
 - Обучить дерево решений на этой выборке, используя на каждом разбиении случайное подмножество признаков.

2. Итоговый прогноз: усреднение (регрессия) или голосование (классификация).

Свойства:

- Высокая точность и устойчивость к переобучению.
- Редко требует настройки гиперпараметров (важны число деревьев B и размер подмножества признаков m).
- **ООВ-оценка:** Каждое дерево обучается на ~63% объектов, остальные 37% (ООВ) могут использоваться для оценки качества этого дерева. Усредняя ООВ-оценки по всем деревьям, получаем оценку обобщающей способности всего леса **без отдельной валидационной выборки**.
- Устойчивость к пропущенным значениям и выбросам.
- Возможность оценивать важность признаков (по тому, насколько уменьшается критерий при разбиении по данному признаку, усреднённого по всем деревьям).

20. Boosting and gradient boosting. Main idea, gradient derivation.

Бустинг (Boosting) — ансамблевый метод, где базовые модели обучаются **последовательно**, каждая новая модель старается исправить ошибки предыдущих.

Основная идея: Итоговая модель — взвешенная сумма слабых моделей (обычно неглубоких деревьев):

$$F_T(x) = \sum_{t=1}^T \rho_t h_t(x)$$

где $h_t(x)$ — слабая модель (базовый алгоритм), ρ_t — её вес.

Градиентный бустинг (Gradient Boosting): Общий фреймворк, где каждая новая модель обучается на **псевдо-остатках** — градиентах функции потерь относительно предсказаний текущего ансамбля.

Вывод (Gradient Boosting для произвольной дифференцируемой функции потерь):

1. Инициализация ансамбля константой: $F_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma)$.
2. Для $t = 1$ до T :

- Вычисление псевдо-остатков для каждого объекта:

$$r_{it} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{t-1}(x)}$$

Это антиградиент функции потерь по предсказаниям модели.

- Обучение базовой модели $h_t(x)$ (например, дерева) на данных $\{(x_i, r_{it})\}$, т.е. модель учится предсказывать эти остатки.

- Нахождение оптимального шага ρ_t (может быть включено в построение дерева или найдено линейным поиском):

$$\rho_t = \arg \min_{\rho} \sum_{i=1}^n L(y_i, F_{t-1}(x_i) + \rho h_t(x_i))$$

- Обновление ансамбля: $F_t(x) = F_{t-1}(x) + \rho_t h_t(x)$.

Интерпретация: Каждое новое дерево "дотягивает" предсказания в направлении, противоположном градиенту ошибки, что соответствует градиентному спуску в **пространстве функций**.

Популярные реализации: XGBoost, LightGBM, CatBoost.

21. Matrix calculus and matrix derivatives. How to get the derivative of matrix/dot product, e.g.: $a^T x$, Ax .

Основные правила матричного дифференцирования (скаляр по вектору/матрице):

1. Производная линейной формы: $f(x) = a^T x = \sum_i a_i x_i$.

$$\frac{\partial(a^T x)}{\partial x} = a$$

2. Производная квадратичной формы: $f(x) = x^T A x$.

$$\frac{\partial(x^T A x)}{\partial x} = (A + A^T)x$$

Если A симметрична, то $\frac{\partial}{\partial x} = 2Ax$.

3. Производная произведения матрицы на вектор: $f(x) = Ax$ (здесь f — вектор).

Якобиан: $\frac{\partial(Ax)}{\partial x} = A$.

4. Производная нормы: $f(x) = \|x\|_2^2 = x^T x$.

$$\frac{\partial(x^T x)}{\partial x} = 2x$$

5. Производная MSE в линейной регрессии:

$$L(w) = \|Y - Xw\|_2^2 = (Y - Xw)^T (Y - Xw)$$

$$\nabla_w L = -2X^T (Y - Xw)$$

Вывод: $\frac{\partial}{\partial w} (Y - Xw)^T (Y - Xw) = -2X^T (Y - Xw)$.

Полезное правило: Если $f(x) = g(h(x))$, то $\frac{\partial f}{\partial x} = \frac{\partial g}{\partial h} \cdot \frac{\partial h}{\partial x}$ (цепное правило).

22. Backpropagation, chain rule.

Обратное распространение ошибки (Backpropagation) — алгоритм эффективного вычисления градиентов функции потерь по всем параметрам (весам) нейронной сети, основанный на **цепном правиле (chain rule)** дифференцирования.

Идея: Нейронная сеть — это композиция многих функций (слоёв). Прямой проход: $\hat{y} = f_L(\dots f_2(f_1(x, w_1), w_2) \dots, w_L)$. Функция потерь $L(\hat{y}, y)$. Нужно найти $\frac{\partial L}{\partial w_l}$ для всех слоёв l .

Алгоритм:

1. **Прямой проход (Forward Pass):** Вычисляются и сохраняются все промежуточные значения активаций $a_l = f_l(z_l)$, где $z_l = W_l a_{l-1} + b_l$.

2. **Обратный проход (Backward Pass):**

- Вычисляется градиент потерь по выходу сети: $\delta_L = \frac{\partial L}{\partial \hat{y}}$.
- Для каждого слоя $l = L, L - 1, \dots, 1$ вычисляется:
- Градиент по взвешенной сумме (логиту) слоя l : $\delta_l = \frac{\partial L}{\partial z_l}$. Используется цепное правило:

$$\delta_l = \frac{\partial L}{\partial z_l} = \left(\frac{\partial L}{\partial z_{l+1}} \frac{\partial z_{l+1}}{\partial a_l} \right) \cdot \frac{\partial a_l}{\partial z_l} = (W_{l+1}^T \delta_{l+1}) \odot f'_l(z_l)$$

где $\odot$ — поэлементное умножение, f'_l — производная функции активации.

- Градиенты по параметрам слоя:

$$\frac{\partial L}{\partial W_l} = \delta_l \cdot a_{l-1}^T, \quad \frac{\partial L}{\partial b_l} = \delta_l$$

Цепное правило (Chain Rule): Если $y = g(u)$ и $u = h(x)$, то $\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$. В многомерном случае — умножение якобианов.

Преимущество backpropagation: Вычисление градиентов происходит за время, сравнимое с прямым проходом, что позволяет обучать сети с миллионами параметров.

23. Neural network concept. Fully-Connected layer (FC). Logistic regression as simple NN. XOR problem.

Нейронная сеть (Neural Network, NN) — вычислительная модель, состоящая из взаимосвязанных обрабатывающих элементов (нейронов), организованных в слои.

Полносвязный слой (Fully-Connected, FC, Dense): Каждый нейрон этого слоя соединён со всеми выходами предыдущего слоя. Операция: $z = W a_{\text{in}} + b$, где W — матрица весов, a_{in} — вектор входных активаций, b — вектор смещений. Затем применяется функция активации: $a_{\text{out}} = f(z)$.

Логистическая регрессия как простейшая нейронная сеть: Это однослойная сеть (один FC-слой) с одним выходным нейроном и сигмоидной функцией активации. Предсказывает

вероятность $P(y = 1|x)$. Функция потерь — логистический лосс (бинарная кросс-энтропия).

Проблема XOR: Одна из первых демонстраций ограниченности линейных моделей. Для данных XOR (исключающее ИЛИ) два класса не являются линейно разделимыми на плоскости. Одна прямая (гиперплоскость) не может разделить точки $(0,0) \rightarrow 0$, $(1,1) \rightarrow 0$ и $(0,1) \rightarrow 1$, $(1,0) \rightarrow 1$.

Решение проблемы XOR: Введение **скрытого слоя** (нелинейного преобразования). Двухслойная сеть (с нелинейной активацией, например, сигмоидой) может решить XOR. Это показало необходимость многослойных (глубоких) сетей для решения нелинейных задач.

24. Losses for NNs: logistic loss, cross-entropy.

Функции потерь измеряют, насколько предсказания сети отличаются от истинных меток.

Логистический лосс (Logistic Loss, бинарная кросс-энтропия): Используется для **бинарной** классификации (два класса, метки 0 и 1). Для одного объекта:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

где $\hat{y} = \sigma(z) \in (0, 1)$ — предсказанная вероятность класса 1.

Кросс-энтропия (Cross-Entropy, категориальная кросс-энтропия): Используется для **многоклассовой** классификации ($K > 2$). Часто в сочетании с функцией активации **softmax** на выходном слое. Для одного объекта:

$$L = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

где y — one-hot вектор истинной метки (1 на позиции правильного класса, 0 остальные), $\hat{y} = \text{softmax}(z)$ — вектор предсказанных вероятностей классов.

Softmax: $\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$. Преобразует логиты z_k в вероятности.

Связь с MLE: Минимизация кросс-энтропии эквивалентна максимизации логарифмического правдоподобия для многоклассового классификатора.

25. Activation functions, their impact on the network, computational complexity. Softmax and LogSoftmax activations, numerical stability.

Функции активации вводят нелинейность в нейронную сеть, позволяя ей приближать сложные функции.

Основные функции:

- **Sigmoid (Логистическая):** $\sigma(x) = \frac{1}{1+e^{-x}}$. Вывод в $(0,1)$. Используется на выходном слое для бинарной классификации. **Проблемы:** Затухание градиентов (производная мала при больших

$|x|$), выход не центрирован относительно нуля.

- **Tanh:** $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$. Вывод в $(-1, 1)$. Центрирована, но также страдает затуханием градиентов.
- **ReLU (Rectified Linear Unit):** $\text{ReLU}(x) = \max(0, x)$. Наиболее популярна. Быстрая в вычислении, не насыщается в положительной области. **Проблема:** "Умирающие ReLU" (градиент равен 0 при $x < 0$).
- **Leaky ReLU:** $\text{LeakyReLU}(x) = \max(\alpha x, x)$, α мало (например, 0.01). Решает проблему "умирания".
- **ELU (Exponential Linear Unit):** Плавная версия, улучшает обучение.

Softmax: Используется **только на выходном слое** для многоклассовой классификации.

Преобразует логиты в вероятности. Вычислительно дорогая из-за экспонент и суммирования по всем классам.

LogSoftmax: $\text{LogSoftmax}(x)_i = x_i - \log\left(\sum_{j=1}^K e^{x_j}\right)$. Вычисляет логарифм вероятности напрямую. Часто используется вместе с функцией потерь **NLLLoss (Negative Log Likelihood Loss)** для большей **численной устойчивости**.

Проблема численной устойчивости softmax: Экспоненты e^{x_j} могут быть очень большими (переполнение) или очень малыми (исчезновение). **Приём:**

$$\text{softmax}(x)_i = \frac{e^{x_i - \max(x)}}{\sum_j e^{x_j - \max(x)}}$$

Вычитание максимума не меняет результат, но гарантирует, что все экспоненты не больше 1, предотвращая переполнение. LogSoftmax реализуется аналогично.

26. Optimization methods in Deep Learning. Gradient descent, SGD, it upgrades: Momentum, RMSProp, Adam.

Градиентный спуск (Gradient Descent, GD): $w_{t+1} = w_t - \eta \nabla Q(w_t)$, где η — скорость обучения, ∇Q — градиент функции потерь по **всем** обучающим данным.

- **Недостаток:** Вычислительно затратен на больших данных.

Стохастический градиентный спуск (Stochastic Gradient Descent, SGD): На каждом шаге градиент оценивается по **одному случайному** объекту (или по **мини-батчу**). Более быстрый, шумный, но часто приводящий к лучшим обобщающим способностям.

Методы улучшения SGD (оптимизаторы):

- **Momentum (Импульс):** Накопление "инерции" от предыдущих градиентов.

$$v_{t+1} = \gamma v_t + \eta \nabla Q(w_t), \quad w_{t+1} = w_t - v_{t+1}$$

Помогает быстрее проходить плато и долины, уменьшает осцилляции.

- **RMSProp:** Адаптивная настройка скорости обучения для каждого параметра. Делит градиент на корень из скользящего среднего квадратов градиентов.

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)g_t^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

Где $g_t = \nabla Q(w_t)$. Уменьшает скорость обучения для часто меняющихся параметров.

- **Adam (Adaptive Moment Estimation):** Комбинация Momentum и RMSProp. Считает скользящее среднее градиентов (первый момент, m_t) и их квадратов (второй момент, v_t), затем применяет коррекцию смещения.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

Один из самых популярных оптимизаторов.

27. Regularization in Deep Learning: Dropout, Batch Normalization. Differences in training and evaluation stages.

Регуляризация в глубоком обучении — методы для борьбы с переобучением в нейронных сетях.

Dropout:

- **Идея:** На этапе обучения на каждом прямом проходе случайным образом "отключает" (обнуляет) с вероятностью p выходы некоторых нейронов (кроме выходного слоя). Это предотвращает ко-адаптацию нейронов, заставляет сеть быть более устойчивой.
- **Реализация:** Умножение выхода слоя на маску из случайных бинарных переменных.
- **На этапе обучения:** Активирован. Масштабирование выхода не требуется в современных реализациях (inverted dropout).
- **На этапе инференса (evaluation):** Dropout отключается. Веса умножаются на p (или выходы умножаются на p), чтобы сохранить математическое ожидание активации. В библиотеках это обычно делается автоматически.

Batch Normalization (BN):

- **Идея:** Нормализует активации внутри мини-батча для каждого нейрона (приводит к нулевому среднему и единичной дисперсии), затем масштабирует и сдвигает с помощью обучаемых параметров γ и β .

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

- **Преимущества:** Ускоряет сходимость, позволяет использовать более высокие скорости обучения, снижает чувствительность к инициализации весов, оказывает слабый регуляризующий эффект (за счёт шума от статистик батча).
- **На этапе обучения:** Вычисляются текущие среднее μ_B и дисперсия σ_B^2 по мини-батчу, обновляются скользящие средние μ_{global} и σ_{global}^2 .
- **На этапе инференса:** Используются фиксированные скользящие средние μ_{global} и σ_{global}^2 , а не статистики батча.

28. Vanilla Recursive NN cell. Backpropagation through RNN. Vanishing gradient problem. Potential solutions.

Рекуррентная нейронная сеть (RNN) — архитектура для обработки последовательностей (текст, временные ряды). **Vanilla RNN cell** (простая версия):

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

где h_t — скрытое состояние в момент t , x_t — вход в момент t .

Обратное распространение во времени (Backpropagation Through Time, BPTT):

Раскручиваем RNN в вычислительный граф на T шагов и применяем стандартный backpropagation. Градиенты по параметрам суммируются по всем шагам.

Проблема затухающего градиента (Vanishing Gradient): При умножении многих матриц $\frac{\partial h_t}{\partial h_{t-1}}$ (якобианы) при BPTT градиенты могут экспоненциально затухать (или взрываться). Особенно сильно в vanilla RNN, так как производная $\tanh$ по модулю < 1 . Это мешает сети обучать долгосрочные зависимости.

Решения:

- **Использование архитектур с gates: LSTM (Long Short-Term Memory) и GRU (Gated Recurrent Unit).** Они имеют специальные вентили (gate), которые регулируют поток информации, позволяя градиентам течь более свободно.
- **Инициализация весов** (например, ортогональная).
- **Использование активаций ReLU** (может помочь, но не полностью решает проблему).
- **Снижение длины последовательностей** (усечение BPTT).
- **Нормализация градиентов** (gradient clipping) для борьбы с взрывом градиентов.

29. LSTM/GRU, memory concept, gates ideas.

LSTM (Long Short-Term Memory): Разработан для решения проблемы затухающего градиента. Вводит **ячейку памяти (cell state) C_t** , которая проходит через всю последовательность практически без изменений, и **ворота (gates)**, регулирующие поток информации.

Основные компоненты LSTM-ячейки:

1. **Forget gate (ворота забывания):** $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$. Решает, какую информацию выбросить из состояния ячейки.
2. **Input gate (входные ворота):** $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$. Решает, какую новую информацию записать в ячейку.
3. **Candidate cell state:** $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$. Потенциальные обновления для состояния ячейки.
4. **Обновление cell state:** $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$. Забываем часть старого состояния и добавляем новое.
5. **Output gate (выходные ворота):** $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$.
6. **Новое hidden state:** $h_t = o_t \odot \tanh(C_t)$. Определяет, какая информация из состояния ячейки будет передана дальше как выход.

GRU (Gated Recurrent Unit): Упрощённая версия LSTM с двумя воротами.

1. **Update gate (ворота обновления):** $z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$. Определяет, какая доля старого состояния сохранится.
2. **Reset gate (ворота сброса):** $r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$. Определяет, какая доля старого состояния используется для вычисления нового кандидата.
3. **Candidate activation:** $\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t] + b)$.
4. **Новое hidden state:** $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$. Интерполяция между старым состоянием и кандидатом.

Обе архитектуры позволяют эффективно обучать долгосрочные зависимости.

30. Matrix convolution. Convolutional layer, backpropagation through it. Hyperparameters of Convs. 1x1 convolutions, comparison to FC layers. Max/Average Pooling.

Свёртка (Convolution) в контексте НС: Операция, в которой ядро (фильтр) скользит по входному изображению (или карте признаков), вычисляя скалярное произведение в каждой позиции.

Свёрточный слой (Conv layer):

- **Вход:** Тензор размера $C_{\text{in}} \times H \times W$ (каналы, высота, ширина).
- **Выход:** Тензор размера $C_{\text{out}} \times H' \times W'$. Каждый выходной канал — результат свёртки входных каналов с соответствующим фильтром и суммирования по входным каналам.
- **Параметры:** Набор фильтров (ядёр). Каждый фильтр — тензор $C_{\text{in}} \times K \times K$ (для квадратного ядра).

Гиперпараметры свёрточного слоя:

- **Размер ядра (Kernel size):** K (например, 3x3).

- **Число фильтров (Number of filters):** C_{out} .
- **Шаг (Stride):** S — шаг, с которым ядро скользит по входу. Уменьшает размерность выхода.
- **Дополнение нулями (Padding):** P — добавление нулей по краям входа для сохранения размерности.

Формула для размерности выхода: $H' = \lfloor \frac{H+2P-K}{S} \rfloor + 1$.

Обратное распространение в свёрточном слое: Градиенты по весам фильтров вычисляются с помощью операции, называемой **обратной свёрткой (transposed convolution)** или просто свёрткой с входными градиентами. Градиенты по входу — это свёртка градиентов по выходу с повернутым на 180° ядром.

Свёртка 1x1: Фильтр размера $1 \times 1 \times C_{\text{in}}$. Не учитывает пространственный контекст, но работает по каналам. Позволяет:

- Уменьшать или увеличивать число каналов (C_{out}).
- Вводить дополнительную нелинейность (с активацией).
- Осуществлять "дешёвое" перемножение каналов.

Сравнение с FC-слоем: Свёртка 1x1 эквивалентна полносвязному слою, применяемому в каждой пространственной позиции независимо и с разделением весов (weight sharing) по позициям. Это значительно уменьшает число параметров.

Слои подвыборки (Pooling): Уменьшают пространственные размеры, обеспечивая инвариантность к малым сдвигам и вращениям.

- **Max Pooling:** Берёт максимум по окну. Наиболее популярен.
- **Average Pooling:** Берёт среднее по окну.
- **Гиперпараметры:** Размер окна (например, 2x2) и шаг (обычно равен размеру окна).

31. Main ideas of AlexNet, VGG, Inception (GoogLeNet), ResNet architectures.

- **AlexNet (2012):** Глубокая CNN, выигравшая ImageNet. Ключевые идеи: использование ReLU для ускорения обучения, Dropout для регуляризации, перекрывающийся Max Pooling, обучение на нескольких GPU. Показала эффективность глубоких свёрточных сетей.
- **VGG (2014):** Демонстрация важности **глубины**. Использует только маленькие свёртки 3x3 (что эквивалентно большой свёртке с меньшим числом параметров) и Max Pooling 2x2. Архитектура простая и однородная (VGG16, VGG19). Главный вывод: увеличение глубины при небольших ядрах улучшает качество.
- **Inception (GoogLeNet, 2014):** Идея **Inception module** — параллельное применение свёрток разных размеров (1x1, 3x3, 5x5) и Max Pooling, а затем конкатенация результатов. Это позволяет сети выбирать оптимальный масштаб признаков. Использует **1x1 свёртки для уменьшения размерности** перед дорогими 3x3 и 5x5. Введение **вспомогательных классификаторов** в промежуточных слоях для борьбы с затуханием градиентов.

- **ResNet (2015):** Решение проблемы **деградации** (degradation) очень глубоких сетей: после определённой глубины точность на training set начинает падать (не из-за переобучения). Введение **остаточных блоков (residual blocks)** с **skip-connections**:

$$y = F(x, \{W_i\}) + x$$

где x — вход блока, F — набор слоёв (например, две свёртки 3×3). Сеть учится остаточному отображению $F(x) = y - x$, что упрощает обучение. Позволила строить сети в сотни и тысячи слоёв. Использует **Batch Normalization** после каждого свёрточного слоя.

32. Geometrical methods in ML. Clustering problem. *IsoMap, LLE, DBSCAN, k-means, t-SNE*

Кластеризация — задача unsupervised learning: разбить множество объектов на группы (кластеры) так, чтобы объекты внутри кластера были похожи, а объекты из разных кластеров — различны.

K-means: Итеративный алгоритм.

1. Инициализация центроидов μ_k (случайно или по правилу).
2. **Назначение:** Каждый объект приписывается к ближайшему центроиду.
3. **Обновление:** Центроиды пересчитываются как среднее объектов их кластера.
4. Повторение шагов 2-3 до сходимости.

Минимизирует внутрикластерную дисперсию. Чувствителен к инициализации, требует задания числа кластеров K , предполагает сферические кластеры.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise): Кластеризация на основе плотности.

- **Параметры:** ϵ (радиус окрестности) и $minPts$ (минимальное число точек для образования кластера).
- **Точки классифицируются на:**
- **Ядровые (core):** В ϵ -окрестности не менее $minPts$ точек.
- **Граничные (border):** Попадают в окрестность ядровой, но сами не ядровые.
- **Шумовые (noise):** Все остальные.
- **Алгоритм:** Начиная с произвольной необработанной ядровой точки, рекурсивно добавляет все достижимые по плотности точки (ядровые и граничные), формируя кластер.
- **Преимущества:** Не требует задания числа кластеров, находит кластеры произвольной формы, устойчив к выбросам.

IsoMap (Isometric Mapping): Нелинейный метод снижения размерности, основанный на **многообразиях**. Сохраняет **геодезические расстояния** (расстояния вдоль многообразия), а не евклидовы. Алгоритм:

1. Строит граф соседства (k ближайших соседей).
2. Вычисляет матрицу геодезических расстояний (кратчайшие пути в графе).
3. Применяет MDS (Multidimensional Scaling) к этой матрице для получения низкоразмерного представления.

LLE (Local Linear Embedding): Также метод снижения размерности для многообразий.

Предполагает, что каждый объект и его ближайшие соседи лежат на локально линейном участке многообразия. Находит веса, линейно восстанавливающие объект из соседей в исходном пространстве, затем сохраняет эти веса в низкоразмерном пространстве.

t-SNE (t-distributed Stochastic Neighbor Embedding): Метод визуализации данных в 2D/3D.

Преобразует сходства между объектами (евклидовы расстояния) в совместные вероятности и старается минимизировать расхождение Кульбака-Лейблера между распределением вероятностей в исходном и низкоразмерном пространствах. Хорошо сохраняет локальную структуру кластеров.

Теоретический минимум

1. Supervised learning problem statement.

Супервизированное обучение — задача построения модели $f : X \rightarrow Y$ на основе размеченной выборки $\{(x_i, y_i)\}_{i=1}^n$, где x_i — вектор признаков, y_i — целевая переменная (ответ). Цель — минимизировать функцию потерь $L(f(x), y)$ на новых данных. Примеры: регрессия ($y \in \mathbb{R}$), классификация ($y \in \{C_1, \dots, C_k\}$).

2. Unsupervised learning problem statement. Provide at least two example problems.

Обучение без учителя — задача нахождения структуры в данных $\{x_i\}_{i=1}^n$ без целевых переменных. Примеры:

1. **Кластеризация:** Разделение данных на группы (кластеры) похожих объектов.
2. **Снижение размерности:** Представление данных в пространстве меньшей размерности с сохранением важной информации.

3. What is i.i.d. data?

i.i.d. (independent and identically distributed) — объекты выборки независимы и одинаково распределены. Это стандартное предположение во многих методах ML, упрощающее анализ. На практике строго выполняется редко.

4. How does a Naive Bayesian Classifier work? Why is it naive?

Алгоритм см. в основном ответе (п.5). "Наивным" называется из-за предположения о **условной независимости признаков при данном классе**, которое редко выполняется в реальности.

5. Linear regression model for MSE minimization problem. Write down the formula and the derivative of the loss function w.r.t. weights.

Модель: $\hat{y} = Xw$. Функция потерь: $L(w) = \frac{1}{n} \|Y - Xw\|^2$. Производная: $\nabla_w L = -\frac{2}{n} X^T (Y - Xw)$.

6. Write down a gradient descent step. How to adjust it for huge datasets?

Шаг градиентного спуска: $w^{(t+1)} = w^{(t)} - \eta \nabla Q(w^{(t)})$. Для больших наборов данных используют **стохастический градиентный спуск (SGD)**, где градиент вычисляется по мини-батчу (подмножеству данных), или **мини-батч SGD**.

7. What is the likelihood? Where is Maximum Likelihood Estimation (MLE) usually used?

Функция правдоподобия $L(\theta|X) = P(X|\theta)$ — вероятность наблюдать данные при заданных параметрах. MLE — метод оценки параметров, максимизирующий эту функцию. Используется повсеместно: в линейной регрессии (при нормальном шуме), логистической регрессии, наивном Байесе, многих вероятностных моделях.

8. What is cross-validation? How does the number of folds affect the validation?

Кросс-валидация — метод оценки обобщающей способности модели. K-fold CV: данные разбиваются на K частей; K-1 частей используются для обучения, 1 — для валидации; процесс повторяется K раз. Большее K уменьшает смещение оценки, но увеличивает дисперсию и вычислительные затраты. K=5 или 10 — типичные значения.

9. What is overfitting and underfitting? How to detect them?

- **Переобучение (Overfitting):** Модель слишком сложная, хорошо предсказывает обучающие данные, плохо — новые. **Обнаружение:** Большая разница между качеством на train и validation/test.
- **Недообучение (Underfitting):** Модель слишком простая, плохо предсказывает и обучающие, и новые данные. **Обнаружение:** Низкое качество и на train, и на validation.

10. What is the difference between parameters and hyperparameters? Provide an example for linear models/decision trees.

- **Параметры** — внутренние переменные модели, настраиваемые в процессе обучения (например, веса w в линейной регрессии, структура дерева решений).
- **Гиперпараметры** — настройки алгоритма обучения, задающиеся до обучения (например, коэффициент регуляризации λ в Ridge, глубина дерева, число соседей в kNN).

11. What is a regularization? What is the difference between L1 and L2 regularization in linear models? Is it the only way to constrain the solution?

Регуляризация — добавление штрафа за сложность модели к функции потерь для борьбы с переобучением. **L2** (Ridge) сжимает веса к нулю, **L1** (Lasso) обнуляет некоторые веса, выполняя отбор признаков. Нет, есть и другие способы: Dropout, BatchNorm в нейросетях, ограничение глубины дерева.

12. Does L2 regularization regularize the bias term (w_0 or b)? Why?

Обычно **нет**. Смещение (bias term, w_0) отвечает за сдвиг модели и не связан с масштабом признаков. Его регуляризация может ухудшить качество, поэтому член регуляризации часто добавляется только к весам при признаках.

13. Why is it a good idea to normalize data before applying a linear model?

1. Ускоряет сходимость градиентных методов.
2. Позволяет сравнивать важность признаков по величине весов (после нормализации веса становятся сопоставимыми).
3. Для некоторых моделей (например, kNN, SVM с RBF-ядром) нормализация критически важна, т.к. они чувствительны к масштабу признаков.
4. Помогает работе регуляризации (L1, L2).

14. Provide a linear classification problem statement. What is a margin?

Задача: по данным $\{(x_i, y_i)\}$, $y_i \in \{-1, +1\}$ найти линейную функцию $f(x) = w^T x + b$, такую что $\text{sign}(f(x))$ хорошо предсказывает класс. **Margin (отступ)** для объекта (x_i, y_i) — величина $M_i = y_i f(x_i)$. Положительный margin означает правильную классификацию, величина margin показывает "уверенность".

15. What are precision and recall? How to use them to measure the model quality?

См. основной ответ (п.2). Precision важна, когда стоимость ложноположительных ошибок высока (например, спам-фильтр). Recall важен, когда важно не пропустить положительные объекты (например, диагностика заболеваний). Обе метрики используются вместе, часто через F1-score или ROC-AUC.

16. Assume the dataset for binary classification is imbalanced, so 95% of data belong to the first class. How to adjust the classification quality measures, to work with such data? Why?

Ассигасу в таком случае бесполезна (модель, всегда предсказывающая класс большинства, получит 95% ассигасу). Следует использовать:

- **Precision, Recall, F1-score** для класса меньшинства.
- **ROC-AUC** (инвариантна к дисбалансу, так как рассматривает оба класса).
- **Сбалансированная ассигасу** (среднее ассигасу по классам).

- **Взвешивание объектов** в функции потерь.

Причина: нужно оценивать, насколько хорошо модель предсказывает **оба класса**, а не просто большинство.

17. What is ROC AUC? How to build the ROC curve?

ROC-AUC — площадь под ROC-кривой. **Построение ROC-кривой:**

1. Модель предсказывает вероятность (или score) принадлежности к положительному классу.
2. Объекты сортируются по убыванию вероятности.
3. Порог классификации изменяется от самого высокого значения до самого низкого.
4. Для каждого порога считаются TPR (Recall) и FPR.
5. ROC-кривая — график TPR от FPR.

18. Logistic loss function. How is it related to Maximum likelihood estimation?

См. основной ответ (п.9). Минимизация логистического лосса эквивалентна максимизации правдоподобия в модели логистической регрессии.

19. Support Vector Machine main idea. The optimization functional for linearly separable case.

Основная идея SVM — найти гиперплоскость, максимизирующую зазор (margin) между классами. Для линейно разделимого случая:

$$\begin{cases} \frac{1}{2} \|w\|^2 \rightarrow \min_{w,b} \\ y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, n \end{cases}$$

20. Describe the greedy optimization algorithm for the decision tree.

См. основной ответ (п.16).

21. Why can an unconstrained decision tree achieve zero error on the training set with unique objects?

Потому что алгоритм может продолжать разбивать пространство до тех пор, пока в каждом листе не останется по одному объекту (или все объекты в листе будут одного класса). Таким образом, на тренировочных данных он достигнет 100% ассигасы (или 0 MSE, если предсказывать значение объекта). Это явный признак переобучения.

22. How to assign a class label for the object in the tree leaf in classification?

Присваивается класс, наиболее часто встречающийся среди объектов, попавших в этот лист (мода).

23. How to assign a class label for the object in the tree leaf in regression? Does it depend on the information criterion?

Присваивается среднее значение целевой переменной объектов в листе. В процессе построения дерева критерий (MSE, MAE) влияет на выбор разбиения, но само значение в листе всегда среднее (или медиана для MAE) обучающих объектов в нём.

24. What is bagging?

См. основной ответ (п.18).

25. What is Random Forest? How does it differ from Bagging over decision trees?

Random Forest = Bagging над деревьями + **Random Subspace Method** (выбор случайного подмножества признаков при каждом разбиении). Это уменьшает корреляцию между деревьями больше, чем обычный бэггинг, что ведёт к большему снижению дисперсии ансамбля.

26. How are base algorithms being trained in gradient boosting?

Базовые алгоритмы (обычно неглубокие деревья) обучаются **последовательно**. Каждое новое дерево обучается на **псевдо-остатках** — градиентах функции потерь относительно предсказаний текущего ансамбля. Таким образом, каждое дерево корректирует ошибки предыдущих.

27. How does backpropagation work in neural networks? What will be vector by vector derivative?

См. основной ответ (п.22). Векторная производная — это якобиан. В backpropagation используются цепное правило и предварительно вычисленные промежуточные значения.

28. How does the Convolutional layer work? What is the convolution operation?

См. основной ответ (п.30). Свёртка — это скалярное произведение ядра (фильтра) с патчем входного изображения, повторяемое для всех позиций.

29. Why fully connected (dense) networks are not the best choice to work with image data? Why do CNNs perform better?

- **FC-сети:**
- Число параметров огромно (например, для изображения $224 \times 224 \times 3 \rightarrow$ входной слой 150528 нейронов).
- Нет учёта **пространственной локальности** (близкие пиксели часто связаны).
- Нет инвариантности к малым сдвигам.
- **CNN лучше:**

- **Разделение весов (weight sharing):** Один фильтр применяется ко всему изображению, резко сокращая число параметров.
- **Локальные связи:** Свёрточные ядра захватывают локальные паттерны (границы, текстуры).
- **Иерархия признаков:** Глубокие слои комбинируют низкоуровневые признаки в высокоуровневые.
- **Инвариантность к сдвигу** (частично обеспечивается пулингом).

30. How does basic RNN (Vanilla RNN) work?

См. основной ответ (п.28). Формула: $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$.

31. How does dropout work?

См. основной ответ (п.27). На обучении случайно "выключаются" нейроны с вероятностью p . На инференсе все нейроны активны, веса масштабируются на p .

32. How do dropout and batch normalization change their behaviour on the inference stage?

- **Dropout:** Отключается. Чтобы сохранить математическое ожидание выхода, веса умножаются на p (или выходы умножаются на p), что делается автоматически в современных фреймворках.
- **Batch Normalization:** Вместо статистик текущего батча используются фиксированные скользящие средние μ_{global} и σ_{global}^2 , накопленные во время обучения.

33. What is the problem statement for the Principal Component Analysis?

Задача: для центрированных данных $X \in \mathbb{R}^{n \times p}$ найти ортонормированные векторы (главные компоненты) $u_1, \dots, u_k$, такие что проекции данных Xu_j имеют максимальную выборочную дисперсию, при условии ортогональности u_j . Это эквивалентно поиску наилучшей (в смысле MSE) линейной аппроксимации данных подпространством меньшей размерности k .